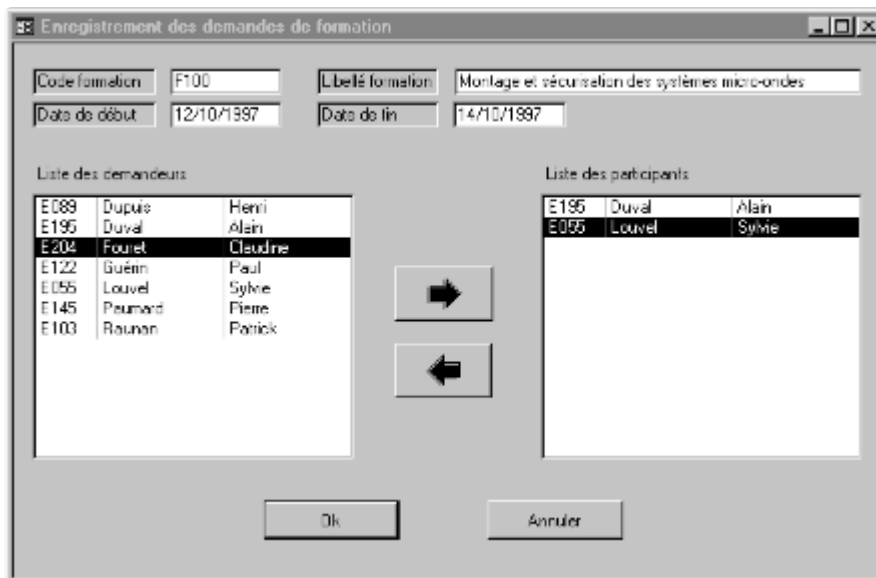


Étude de cas : gestion des demandes de formation



Contexte

Les pompiers et personnels de secours doivent suivre régulièrement des formations. On veut développer une **application C#** de gestion des demandes de formation, autour d'une base de données.

📘 Fonctionnement général

- Les demandes écrites doivent parvenir au centre de formation **10 jours avant** le début de la formation.
- Le lendemain de cette date limite, les demandes reçues sous forme papier sont **toutes enregistrées**.

Une maquette a été validée avec les utilisateurs. L'écran ci-dessus s'affiche une fois que l'utilisateur a choisi la formation à traiter.

Fonctionnement de l'écran

Action	Effet
Chargement de la fenêtre	La liste des demandeurs est extraite de la BDD et copiée dans un objet <code>ListeOP</code> (matricule, nom, prénom)
Clic souris sur un opérateur	Sélection dans l'une des deux listes
Bouton Flèche droite	Le demandeur sélectionné passe dans les participants (<code>clic_flèche_droite()</code>)

Action	Effet
Bouton Flèche gauche	Le participant sélectionné repasse dans les demandeurs (clic_flèche_gauche())

Les deux listes s'appellent **Demandeurs** et **Participants**. Ce sont toutes deux des **instances** de la classe `ListeOP`.

La classe `ListeOP`

```

classe ListeOP
privé
    éléments : tableau[1..max] de OP    // OP = opérateur (salarié de
l'entreprise)
    isel      : entier                  // indice de l'élément sélectionné
    ilibres   : entier                  // indice du premier élément libre
    Procédure rafraîchir()              // actualise l'affichage de la
liste
    ...

public
    Procédure ajouter(donnée UnOp : OP) // ajoute UnOp à la liste + met à
jour l'affichage
    Fonction vide() : booléen           // vrai si la liste est vide
    Fonction récupérer() : OP           // retourne l'opérateur
sélectionné
    Procédure retirer()                  // supprime l'opérateur
sélectionné + met à jour l'affichage
    ...
fin classe

```

Informations utiles



1. Accès au nom d'un opérateur

Le nom est accessible via le champ `nom` :

```

Var UnOp : OP
...
UnOp.nom <- "dupont"
afficher(UnOp.nom)    // affiche « dupont »

```



2. Principe de `ajouter(UnOp)`

- Au départ, `éléments` est **trié par ordre alphabétique des noms** et il reste au moins une place libre (`ilibres` reste toujours inférieur à `max`).
- Le traitement consiste à :
 - insérer `UnOp` dans `éléments` **à une place qui conserve l'ordre de tri** ;
 - actualiser l'affichage de la liste.



3. Portée des objets

Les objets `Demandeurs` et `Participants` sont accessibles à l'intérieur des procédures `clic_flèche_droite()` et `clic_flèche_gauche()` .

Travail à faire

1. Écrire l'algorithme de la procédure `clic_flèche_gauche()` en utilisant les méthodes de la classe `ListeOP` .
2. Ajouter à la classe `ListeOP` une méthode retournant le **nombre d'éléments** contenus dans la liste `éléments` .

Réponse

1 — `clic_flèche_gauche()`

Idée : on déplace le participant sélectionné de `Participants` vers `Demandeurs` .

```
Procédure clic_flèche_gauche()
    Var UnOp : OP

    // Rien à faire si la liste des participants est vide
    Si Non Participants.vide() Alors
        Demandeurs.ajouter(Participants.récupérer())
        Participants.retirer()
    Fin Si
Fin Procédure
```

2 — Méthode `nombre_elements()`

Méthode **publique** de la classe `ListeOP` : elle utilise directement l'attribut privé `ilibres` , sans préfixe.

```
Fonction nombre_elements() : entier
    // ilibres = indice du premier élément libre, donc nombre d'éléments =
    ilibres - 1
    retourner ilibres - 1
Fin Fonction
```

Exemple d'appel : `Demandeurs.nombre_elements()`